5. PROJECT DESIGN PHASE

Automating Employee Data Import & Relationship Mapping Using Import Sets and Dot Walking

1. DESIGN OBJECTIVES

This document constitutes the formal Design Phase output for the ServiceNow Automation Project. Its purpose is to translate the approved business requirements — as documented in the Requirement Analysis — into a precise technical blueprint that will govern all configuration and build activities performed during the subsequent Development Phase. The design specification herein defines the architectural structure, data models, component interactions, interface configurations, and governance constraints within which the solution will be constructed.

- Simplicity over complexity: The design leverages exclusively native ServiceNow platform capabilities — Import Sets, Transform Maps, and form configuration — without introducing custom scripting, scoped applications, or external integrations that would increase maintenance burden.
- Data integrity by design: All reference field relationships, particularly the employee-to-manager linkage, are resolved at transformation time via the platform's native reference resolution engine, ensuring that data relationships are structurally enforced at the record level rather than managed by convention.
- Non-invasive presentation: The Incident form augmentation employs read-only dot-walked fields, ensuring that contextual user data is surfaced to service desk agents without introducing any write path to the [sys_user] table from the Incident form, preserving data governance boundaries.
- Reusability: All Transform Map configurations will be persistable and reusable across future import cycles, eliminating the need to rebuild the mapping configuration for each data load.

2. SOLUTION ARCHITECTURE OVERVIEW

2.1 Architectural Principles

The solution is designed as a three-layer ServiceNow-native architecture, operating entirely within the Global application scope. Each layer serves a distinct and bounded responsibility, and data flows unidirectionally through the layers — from ingestion through transformation to persistence and presentation. No layer bypasses another, and no layer introduces a write path in the reverse direction. The layered model is defined in the following table.

1	**Data Ingestion Layer** `Import Sets`	Receives raw external data from the manually uploaded CSV file and stages it in a dedicated temporary table ([u_import_employee]) without modification. This layer is solely responsible for data capture and structural preservation of the source payload. No transformation, validation, or target-table interaction occurs at this layer.
2	**Transformation Layer** `Transform Maps`	Applies field-level mapping rules to the staged data and writes the transformed records to the target [sys_user] table. This layer is responsible for all resolution logic, including the manager reference relationship, direct field copies, and coalesce-based duplicate prevention. The transformation is executed synchronously on demand by an administrator.
3	**Presentation Layer** `Form Layout & Dot-Walking`	Surfaces related user data on the Incident form by traversing the reference relationship from the Incident's 'Assigned to' field into the [sys_user] record. This layer introduces no data writes and adds no new tables. All field values are derived dynamically at form render time via the platform's native dot-walking mechanism.

2.3 Component Interaction Summary

The following table summarises the interaction contracts between the three architectural layers. Each row defines the mechanism by which data crosses from one component to the next, the directionality of that interaction, and whether the interaction is synchronous or deferred.

From Component	To Component	Interaction Mechanism	Direction	Trigger
External CSV file	`u_import_employee`	File upload via Import Sets Load Data form	→ One-way	Manual — administrator
`u_import_employee`	`sys_user`	Transform Map field mapping + reference resolution	→ One-way	Manual — Transform execution
`sys_user`	Incident form	Dot-walking via `assigned_to` reference field	→ Read-only	Automatic — on user field change

3. DATA ARCHITECTURE & MODEL DESIGN

3.1 Source Data Schema (External CSV)

The external data source will be a CSV-formatted file conforming to the schema defined below. This schema represents the minimum required structure; additional columns in the source file will be ignored by the Transform Map unless explicitly mapped. The schema definition must be communicated to the Data Owner as the authoritative file specification and must not change between import cycles without a corresponding Transform Map update.

Column Header	Logical Data Type	Max Length	Required	Uniqueness Constraint	Notes
`name`	String	100 chars	Yes	No	Full display name. Must match the display name format used in [sys_user] for manager reference resolution to succeed.
`email`	String	100 chars	Yes	Yes (per batch)	Corporate email address. Used as the coalesce (deduplication) key on re-import. Must be unique within the source file.
`department`	String	50 chars	No	No	Department name. Must correspond to an existing Department record in the platform for the reference field to resolve.
`manager`	String	100 chars	No	No	Full name of the reporting manager. Must match the name of an existing or co-imported [sys_user] record exactly.
`phone number`	String	20 chars	No	No	Primary contact number. Stored as a string; no numeric validation is applied during import.

3.2 Staging Table Schema ([u_import_employee])

The Import Set staging table will be auto-generated by the ServiceNow platform upon the first file upload via the Import Sets Load Data form. The platform derives the column names from the CSV file headers, applying a 'u_' prefix to each column name as part of the import_set_row extension pattern. The resulting physical schema is defined below.

Physical Column Name	Derived From (CSV Header)	Platform Type	Nullable	Notes
`u_name`	`name`	String	Yes	Auto-generated on table creation. Stores the raw source value unchanged.
`u_email`	`email`	String	Yes	Auto-generated. Will serve as the basis for the coalesce mapping.
`u_department`	`department`	String	Yes	Auto-generated. Stores the raw department string prior to resolution.
`u_manager`	`manager`	String	Yes	Auto-generated. Stores the raw manager name text; resolution to a reference occurs in the Transformation Layer only.
`u_phone_number`	`phone number`	String	Yes	Auto-generated. Note: the space in the CSV header becomes an underscore in the physical column name.

3.3 Target Table Schema ([sys_user] — Relevant Fields)

Field Name	Platform Type	Source Mapping	Coalesce	Design Notes
`name`	String	`u_name`	No	Direct copy from staging. Serves as the primary display name of the user record.
`email`	String	`u_email`	Yes	Designated coalesce field. On re-import, the transform engine will update an existing record whose email matches rather than creating a duplicate.
`department`	Reference (sys_dept)	u_department`	No	The raw string value from staging is resolved against the [cmn_department] table's Name field. If no match is found, the field will be left blank rather than creating an error.
`phone`	String (Phone)	`u_phone_number`	No	Direct copy. Note the field name difference: the target field is 'phone', not 'phone_number'. This requires explicit field mapping.
`manager`	Reference (sys_user)	u_manager`	No	Self-referencing relationship field. The string value from staging is resolved against the [sys_user] Name field to establish the reference link. See Section 3.4 for the full resolution design.

3.4.2 Reference Resolution Sequence

1	**Read source value** *The transform engine reads the raw text from the u_manager column of the current staging row (e.g., 'Bob Smith').*
2	**Query [sys_user] by display value** *The engine executes an internal lookup against [sys_user] where name = 'Bob Smith' (using the reference field's display value field).*
3	**Evaluate match result** *If exactly one matching record is found, the sys_id of that record is used. If zero or multiple matches are found, the field is left unresolved.*
4	**Write reference to target record** *The resolved sys_id is written to the manager reference field on the employee's newly created or updated [sys_user] record.*
5	**Resolution confirmed** *The employee record's manager field now holds a navigable reference link to the manager's [sys_user] record. The relationship is structurally enforced at the database level.*

4. COMPONENT DESIGN SPECIFICATIONS

4.1 Import Set Table Design

The Import Set staging table will be auto-generated by the platform during the initial file upload. The following specification defines the expected configuration state of this component following successful table creation.

Component Type	Import Set Table (user-created, auto-generated from CSV)
Physical Table Name	u_import_employee
Display Label	import employee
Extends	import_set_row (base ServiceNow import staging parent table)
Application Scope	Global
Persistence	Retained after transformation. Not auto-purged. Serves as an audit record of each import batch.
Record Count	One row per employee in the source CSV file. Minimum four records for acceptance criteria.
Column Generation	Columns auto-derived from CSV headers. Each header becomes a field prefixed with 'u_'.

4.2 Transform Map Design

The Transform Map is the central orchestrating component of the Transformation Layer. The following specification defines its complete configuration, including all field mapping rules and the coalesce strategy.

Component Type	Transform Map
Name	Employee Import Transform
Source Table	u_import_employee (Import Set staging table)
Target Table	sys_user (Core User table)
Run Order	100 (platform default — no ordering adjustment required)
Application Scope	Global
Active	True

4.3 Dot-Walking Configuration Design

The dot-walking design extends the native reference traversal capability of the ServiceNow platform to surface related data from the [sys_user] table directly on the Incident form. No new fields are added to either the [incident] or [sys_user] tables; instead, the form layout is configured to include virtual dot-walked display fields that are dynamically populated at render time.

5. USER INTERFACE DESIGN

5.1 Incident Form Layout Design

The Incident form will be augmented with three read-only fields positioned in the Assignment Information section. The following wireframe description specifies the intended form layout following the configuration change. This representation conveys the structural intent of the design without prescribing an exact visual position, as form rendering may vary slightly across ServiceNow interface themes.

5.2 User Experience Flow Design

The following sequence describes the interaction model for a service desk agent using the configured Incident form. This flow requires no additional agent training beyond awareness that the three fields are auto-populated — no manual data entry in those fields is expected or possible.

1	**Agent opens a new Incident record** *The Incident form loads in its default view. The three dot-walked fields are visible but empty, as no user has been assigned yet.*
2	**Agent populates the 'Assigned to' field** *The agent types a name or uses the reference lookup to select a user from [sys_user]. The user must be one whose record has Department, Email, and Manager populated.*
3	**Platform evaluates the reference change** *Upon selection, the form's native onChange behaviour for reference fields triggers a re-evaluation of all dot-walked dependent fields.*
4	**Dot-walked fields populate automatically** *The three fields — Assigned to Department, Assigned to Email, and Assigned to Manager — are populated with values traversed from the selected user's [sys_user] record. No additional action is required from the agent.*
5	**Agent reads contextual data inline** *The agent can immediately see the assigned user's department, email, and manager without navigating to the User record. This reduces lookup time and supports faster incident triage and escalation decisions.*
6	**Agent saves the Incident** *The Incident is saved with the selected assigned user. The dot-walked fields are not stored as independent Incident field values; they will re-derive from the User record on each subsequent form load.*

6. INTEGRATION DESIGN

Integration Attribute	Design Specification
Integration Type	Manual file upload. No API, middleware, or real-time connection.
Protocol	Not applicable. Data is delivered as a static file through the ServiceNow browser-based Import Sets interface.
Source System	External — any system capable of generating a CSV or Excel file conforming to the source schema defined in Section 3.1.
Trigger Mechanism	Administrator-initiated. An authorised user manually uploads the file and initiates the transformation. No event-driven or scheduled trigger is included in scope.
Frequency	Ad-hoc. The import will be performed as needed, typically for initial data population or when new employee records become available.
Authentication	Implicit. File uploads occur within an active authenticated ServiceNow session. No external system credentials are required, and no API keys or tokens are exchanged.
Error Propagation	Transform errors are surfaced in the Transform History record within the ServiceNow UI. No external notification, alerting, or error queue is designed.
Data Volume	Low volume. The design is optimised for small batches of employee records (tens to hundreds of records). High-volume imports are outside the design scope.

7. SECURITY & ACCESS DESIGN

Component	Read Access	Write / Execute Access	Rationale
Import Set Table [u_import_employee]	import_set_loader	import_set_loader	Restricts staging data access to import administrators. General users and ITIL agents have no access to raw staging data.
Transform Map configuration	import_set_administrator	import_set_administrator	Only import administrators may create, modify, or delete Transform Map c
Transform Map execution (run)	import_set_administrator	import_set_administrator	Execution of the Transform Map writes to th
Incident form — dot-walked fields	All itil-role users	None (read-only)	Dot-walked fields are display-only by platform default. No ACL modification is required; the reand does not require a custom write ACL.
User records [sys_user]	Default platform read access	user_admin role	Write access to user records is governed by the standoperates under the admin session context during execution.

8. PERFORMANCE DESIGN CONSIDERATIONS

8.1 Transform Map Performance

The Transform Map is designed to use exclusively Direct mapping types for all five field mappings. Direct mappings are the most performant mapping type available within the ServiceNow Transform Map engine, as they execute as simple value assignments or reference lookups without invoking the GlideScript engine. The deliberate avoidance of Script Map types — which would require JavaScript execution per row — ensures that the transform can process the target data volume efficiently within normal platform execution bounds.

8.2 Coalesce Lookup Performance

The coalesce strategy specifies the email field as the deduplication key. The [sys_user].email field is indexed by default in standard ServiceNow deployments, which ensures that the coalesce lookup — a query of the form WHERE email = '[value]' executed per imported row — will use an index-based scan rather than a full table scan. The developer must confirm that this index is present on the target instance before the first production import execution.

8.3 Dot-Walking Render Performance

Dot-walked fields on the Incident form are evaluated at form render time using ServiceNow's standard reference field traversal mechanism. The three dot-walked fields each require a single-hop traversal from the Incident record to the [sys_user] record. This traversal pattern is well within the performance parameters of the platform's standard form rendering pipeline and does not require any caching, asynchronous loading, or GlideAjax optimisation for the data volumes described in this project.

9. ASSUMPTIONS & DESIGN CONSTRAINTS

9.1 Design Assumptions

Assumption ID	Assumption Statement	Impact If Invalid	Owner to Validate
DA-01	Manager records either pre-exist in [sys_user] at the time of employee import, or are included in the same import batch and processed before dependent employee rows.	Manager reference fields will be unresolved on affected employee records. A second transform execution will be required to resolve outstanding references.	Data Owner / HR Representative
DA-02	The 'name' field values used in the manager column of the source CSV exactly match the corresponding display name values of the manager's [sys_user] record.	Reference resolution will fail for any manager whose name in the CSV does not exactly match the [sys_user] name field value.	Data Owner / HR Representative
DA-03	The [sys_user].email field is indexed on the target instance, ensuring efficient coalesce lookups during re-import operations.	Coalesce lookups may perform as full table scans on large [sys_user] tables, increasing transform execution time.	ServiceNow Administrator
DA-04	Department values in the source CSV correspond to Department records that already exist in the [cmn_department] table on the platform.	The department reference field will be unresolved on imported user records, leaving the Department field blank.	Data Owner / HR Representative
DA-05	The Incident form being modified is the standard Default view in Global scope, not a customised or scoped-application-specific view.	Form layout changes may not apply to all users if non-standard views are in use. Additional form layout configurations may be required.	ServiceNow Administrator